

React JS Interview Guide (0–3 Years Experience)

Complete React.js interview preparation notes covering fundamentals, advanced concepts, coding questions, architecture, hooks, performance, ecosystem, system design basics, and interview Q&A.

Table of Contents

1. Introduction to React
2. React Features
3. JSX
4. Components
5. Props
6. State
7. Event Handling
8. Conditional Rendering
9. Lists and Keys
10. Forms
11. Lifting State Up
12. Controlled vs Uncontrolled Components
13. React Lifecycle
14. Functional Components vs Class Components
15. Hooks
16. useState
17. useEffect
18. useContext
19. useRef
20. useMemo
21. useCallback
22. Custom Hooks
23. React Router
24. State Management
25. Redux Toolkit
26. Context API
27. Performance Optimization
28. React.memo
29. Lazy Loading
30. Suspense
31. Error Boundaries
32. Portals
33. Higher Order Components
34. Render Props
35. Compound Components
36. Styling in React

37. CSS Modules
 38. Styled Components
 39. Tailwind CSS
 40. API Calls
 41. Axios vs Fetch
 42. Authentication
 43. JWT Authentication
 44. Protected Routes
 45. React Query / TanStack Query
 46. Form Libraries
 47. Testing in React
 48. Jest
 49. React Testing Library
 50. TypeScript with React
 51. Folder Structure
 52. Build Tools
 53. Vite vs CRA
 54. SSR vs CSR
 55. Next.js Basics
 56. React Internals
 57. Virtual DOM
 58. Reconciliation
 59. Diffing Algorithm
 60. React Fiber
 61. Common Interview Questions
 62. Machine Coding Questions
 63. Coding Challenges
 64. System Design for Frontend
 65. Best Practices
 66. React Project Ideas
 67. HR Interview Questions
 68. Final Revision Notes
-

1. Introduction to React

React is a JavaScript library developed by Facebook for building user interfaces.

Features

- Component-based architecture
- Reusable UI components
- Virtual DOM
- One-way data binding
- Fast rendering
- Declarative programming

- Rich ecosystem

Why React?

- Easy to build SPAs
 - High performance
 - Huge community support
 - Reusable components
 - SEO support with Next.js
-

2. JSX

JSX stands for JavaScript XML.

```
const element = <h1>Hello React</h1>;
```

Rules of JSX

- Must return single parent element
- Use className instead of class
- Use camelCase for attributes
- Expressions inside {}

```
const name = 'React';  
return <h1>{name}</h1>;
```

JSX Advantages

- Easy to write
 - Better readability
 - Prevents injection attacks
-

3. Components

Components are reusable building blocks.

Functional Component

```
function Welcome() {  
  return <h1>Hello</h1>;  
}
```

Class Component

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

Types of Components

- Functional
 - Class
 - Pure
 - Higher Order Components
-

4. Props

Props are read-only inputs passed to components.

```
function User(props) {  
  return <h1>{props.name}</h1>;  
}
```

Features

- Immutable
 - Parent to child communication
-

5. State

State is mutable data managed inside components.

```
const [count, setCount] = useState(0);
```

Difference Between Props and State

Props	State
Immutable	Mutable
Passed from parent	Managed inside component
Read-only	Can update

6. Event Handling

```
<button onClick={handleClick}>Click</button>
```

Common Events

- onClick
- onChange
- onSubmit
- onMouseEnter
- onKeyDown

7. Conditional Rendering

Using Ternary

```
{isLoggedIn ? <Home /> : <Login />}
```

Using &&

```
{isAdmin && <AdminPanel />}
```

8. Lists and Keys

```
users.map(user => (  
  <li key={user.id}>{user.name}</li>  
))
```

Why Keys?

- Helps React identify elements
 - Improves rendering performance
-

9. Forms

Controlled Component

```
const [name, setName] = useState('');  
  
<input value={name} onChange={(e)=>setName(e.target.value)} />
```

Uncontrolled Component

```
const inputRef = useRef();
```

10. Lifting State Up

Moving state to common parent component.

Used for sibling communication.

11. Lifecycle Methods

Mounting

- constructor
- render

- componentDidMount

Updating

- shouldComponentUpdate
- componentDidUpdate

Unmounting

- componentWillUnmount
-

12. Hooks

Hooks allow functional components to use state and lifecycle features.

Rules of Hooks

- Call at top level
 - Call only inside React functions
-

13. useState

```
const [count, setCount] = useState(0);
```

Important Points

- Async updates
- Functional updates

```
setCount(prev => prev + 1);
```

14. useEffect

Used for side effects.

```
useEffect(() => {  
  fetchUsers();  
}, []);
```

Dependency Array Cases

No Dependency

Runs every render.

Empty Dependency

Runs once.

With Dependency

Runs when dependency changes.

Cleanup Function

```
useEffect(() => {  
  const timer = setInterval(() => {}, 1000);  
  
  return () => clearInterval(timer);  
}, []);
```

15. useContext

Avoids prop drilling.

```
const UserContext = createContext();
```

```
const user = useContext(UserContext);
```

16. useRef

Used for:

- DOM access
- Persisting mutable values

```
const ref = useRef();
```

17. useMemo

Memoizes expensive calculations.

```
const result = useMemo(() => compute(data), [data]);
```

Use Cases

- Heavy calculations
- Prevent unnecessary recomputation

18. useCallback

Memoizes functions.

```
const memoFn = useCallback(() => {  
  fetchData();  
}, []);
```

Use Cases

- Passing callbacks to child components
- Performance optimization

19. Custom Hooks

```
function useCounter() {  
  const [count, setCount] = useState(0);  
  
  const increment = () => setCount(c => c + 1);  
  
  return { count, increment };  
}
```

20. React Router

Installation

```
npm install react-router-dom
```

Basic Routing

```
<BrowserRouter>  
  <Routes>  
    <Route path="/" element={<Home />} />  
  </Routes>  
</BrowserRouter>
```

Hooks

- useNavigate
- useParams
- useLocation

21. State Management

Options

- Context API
- Redux
- Zustand

- MobX
 - Recoil
-

22. Redux Toolkit

Store

```
configureStore({  
  reducer: {}  
})
```

Slice

```
const counterSlice = createSlice({  
  name: 'counter',  
  initialState: 0,  
  reducers: {  
    increment: state => state + 1  
  }  
});
```

Important Concepts

- Store
 - Reducer
 - Action
 - Dispatch
 - Middleware
-

23. Context API

When to Use

- Theme
- Authentication
- Language

When Not to Use

- Frequently changing large state
-

24. Performance Optimization

Techniques

- React.memo
 - useMemo
 - useCallback
 - Code splitting
 - Lazy loading
 - Virtualization
-

25. React.memo

Prevents unnecessary re-render.

```
export default React.memo(Component);
```

26. Lazy Loading

```
const Home = React.lazy(() => import('./Home'));
```

27. Suspense

```
<Suspense fallback={<Loader />}>  
  <Home />  
</Suspense>
```

28. Error Boundaries

Catches UI errors.

```
class ErrorBoundary extends React.Component {}
```

Cannot Catch

- Event handler errors
 - Async errors
-

29. Portals

Render outside root DOM.

```
ReactDOM.createPortal(child, container)
```

Used for:

- Modals
 - Tooltips
 - Popups
-

30. Higher Order Components (HOC)

A function that takes component and returns enhanced component.

```
const Enhanced = withAuth(Component);
```

31. Render Props

```
<Data render={(data) => <UI data={data} />} />
```

32. Compound Components

Used in reusable component libraries.

Example:

```
<Tabs>
  <Tabs.List />
  <Tabs.Panel />
</Tabs>
```

33. Styling in React

Methods

- CSS
- SCSS
- CSS Modules
- Styled Components
- Tailwind CSS

34. CSS Modules

```
import styles from './App.module.css';
```

35. Styled Components

```
const Button = styled.button`
  color: red;
`;
```

36. Tailwind CSS

Utility-first CSS framework.

```
<div className="flex items-center">
```

37. API Calls

Fetch

```
fetch('/api/users')
```

Axios

```
axios.get('/api/users')
```

38. Axios vs Fetch

Axios	Fetch
Auto JSON transform	Manual
Better error handling	Basic
Interceptors	No

39. Authentication

Types

- JWT
 - Session-based
 - OAuth
-

40. JWT Authentication

Flow

1. Login
2. Backend returns token
3. Store token
4. Send token in headers

```
Authorization: Bearer token
```

41. Protected Routes

```
return isAuthenticated ? <Home /> : <Navigate to="/login" />;
```

42. React Query / TanStack Query

Features

- Caching
- Refetching
- Retry
- Background updates

```
const { data } = useQuery({
  queryKey: ['users'],
  queryFn: fetchUsers
});
```

43. Form Libraries

React Hook Form

```
const { register, handleSubmit } = useForm();
```


Formik

Popular form handling library.

44. Testing in React

Types

- Unit testing
 - Integration testing
 - End-to-end testing
-

45. Jest

```
test('renders text', () => {})
```

46. React Testing Library

```
render(<App />)
```

Best Practice

Test behavior not implementation.

47. TypeScript with React

Props Interface

```
interface Props {  
  name: string;  
}
```

```
const User: React.FC<Props> = ({ name }) => {}
```

48. Folder Structure

Feature Based

```
src/  
├─ features/  
├─ components/  
├─ hooks/  
├─ services/  
└─ utils/
```

49. Build Tools

Common Tools

- Vite
- Webpack
- Babel
- Parcel

50. Vite vs CRA

Vite	CRA
Faster	Slower
ESBuild	Webpack
Modern	Older

51. SSR vs CSR

SSR	CSR
Better SEO	Faster transitions
Server rendering	Client rendering

52. Next.js Basics

Features

- SSR
- SSG
- API routes
- Image optimization
- File-based routing

53. Virtual DOM

Virtual representation of real DOM.

Advantages

- Faster updates
- Efficient rendering

54. Reconciliation

Process of updating DOM efficiently.

55. Diffing Algorithm

React compares old and new Virtual DOM.

Complexity optimized from $O(n^3)$ to $O(n)$.

56. React Fiber

New reconciliation engine.

Features

- Incremental rendering
 - Priority updates
 - Better responsiveness
-

57. Common React Interview Questions

Beginner Level

What is React?

React is a JavaScript library for building user interfaces.

What is JSX?

JSX is syntax extension for JavaScript.

Difference between state and props?

Props are immutable; state is mutable.

What is Virtual DOM?

A lightweight copy of real DOM.

What are hooks?

Hooks allow functional components to use state and lifecycle.

58. Intermediate Interview Questions

Explain useEffect lifecycle.

Runs after render and handles side effects.

Difference between useMemo and useCallback?

- useMemo memoizes values
- useCallback memoizes functions

What is prop drilling?

Passing props deeply through components.

What is Context API?

Global state management solution.

Difference between controlled and uncontrolled components?

Controlled uses React state; uncontrolled uses DOM refs.

59. Advanced Interview Questions

Explain React Fiber.

Fiber enables asynchronous rendering and prioritization.

Explain reconciliation.

React compares trees and updates minimal DOM.

What causes re-renders?

- State updates
- Props updates
- Context updates
- Parent re-renders

How to optimize React app?

- Memoization
 - Lazy loading
 - Code splitting
 - Virtualization
 - Avoid anonymous functions
-

60. Machine Coding Questions

Common Questions

- Todo App
 - Infinite Scroll
 - Debounced Search
 - Drag and Drop
 - Modal Component
 - Accordion
 - Tabs Component
 - Multi-step Form
 - File Upload
 - Chat UI
-

61. Coding Challenges

Debounce Function

```
function debounce(fn, delay) {  
  let timer;  
  
  return (...args) => {  
    clearTimeout(timer);  
  
    timer = setTimeout(() => {  
      fn(...args);  
    }, delay);  
  };  
}
```

62. Throttle Function

```
function throttle(fn, delay) {  
  let allow = true;  
  
  return (...args) => {  
    if (!allow) return;  
  };  
}
```

```
fn(...args);  
allow = false;  
  
setTimeout(() => {  
  allow = true;  
}, delay);  
};  
}
```

63. Frontend System Design Basics

Design Questions

- Design YouTube UI
- Design Chat App
- Design E-commerce Frontend
- Design Notification System

Important Topics

- Caching
- Pagination
- Lazy loading
- Accessibility
- Responsive design
- Error handling

64. Accessibility

Best Practices

- Semantic HTML
 - aria-label
 - Keyboard navigation
 - Screen reader support
-

65. Security Best Practices

Avoid

- XSS
- Storing sensitive data in localStorage

Use

- HTTPS
 - Secure cookies
 - Input sanitization
-

66. React Best Practices

Do

- Use reusable components
- Keep components small
- Use hooks effectively
- Use TypeScript
- Use proper folder structure

Avoid

- Prop drilling
 - Large components
 - Inline functions everywhere
 - Unnecessary re-renders
-

67. Common Mistakes

- Missing dependency array
 - Wrong keys in list
 - Mutating state directly
 - Too many global states
 - Not cleaning useEffect
-

68. React Project Ideas

Beginner

- Todo App
- Weather App
- Notes App

Intermediate

- E-commerce App
- Chat App
- Expense Tracker

Advanced

- Real-time Collaboration App
 - Video Streaming App
 - Social Media App
-

69. HR Interview Questions

Tell me about yourself.

Focus on:

- Skills
- Projects
- Experience
- Achievements

Why React?

Discuss reusable UI, ecosystem, performance.

Explain your project architecture.

Mention:

- Folder structure
- State management
- APIs
- Authentication

- Optimization
-

70. Important React APIs

APIs

- `useId`
 - `useDeferredValue`
 - `useTransition`
 - `forwardRef`
 - `memo`
 - `lazy`
-

71. React 18 Features

Features

- Automatic batching
 - Concurrent rendering
 - Suspense improvements
 - `startTransition`
-

72. `useTransition`

```
const [isPending, startTransition] = useTransition();
```

Used for non-blocking updates.

73. `useDeferredValue`

Used to defer expensive rendering.

```
const deferredValue = useDeferredValue(value);
```

74. forwardRef

```
const Input = forwardRef((props, ref) => {  
  return <input ref={ref} />;  
});
```

75. React.memo vs useMemo vs useCallback

Feature	Purpose
React.memo	Memoize component
useMemo	Memoize value
useCallback	Memoize function

76. Redux Middleware

Examples

- Redux Thunk
- Redux Saga

77. WebSocket in React

```
const socket = new WebSocket(url);
```

Used in:

- Chat apps
- Live updates

78. Environment Variables

```
VITE_API_URL=https://api.com
```

79. Deployment

Platforms

- Vercel
 - Netlify
 - Firebase Hosting
 - AWS
-

80. Important JavaScript Topics for React Interviews

Must Learn

- Closures
 - Promises
 - Async/Await
 - Event Loop
 - Hoisting
 - Debounce
 - Throttle
 - Array methods
 - ES6+
-

81. Top React Interview Tips

Before Interview

- Revise hooks
- Practice machine coding
- Know project deeply
- Practice JavaScript
- Learn optimization

During Interview

- Think aloud
 - Explain tradeoffs
 - Write clean code
 - Handle edge cases
-

82. Rapid Revision Sheet

Hooks

Hook	Use
useState	State
useEffect	Side effects
useContext	Global data
useRef	DOM access
useMemo	Memoize values
useCallback	Memoize functions

83. Frequently Asked Scenario Questions

How to avoid unnecessary re-renders?

- React.memo
- useMemo
- useCallback
- Proper state management

How to share state between components?

- Lifting state
- Context API
- Redux

How to optimize large lists?

- Virtualization
- Pagination

- Infinite scrolling
-

84. React Architecture Tips

Good Architecture

- Separation of concerns
 - Reusable components
 - Feature-based folders
 - API abstraction
 - Reusable hooks
-

85. Important Libraries

Popular Libraries

- React Router
 - Redux Toolkit
 - React Query
 - Axios
 - React Hook Form
 - Framer Motion
 - Material UI
 - Tailwind CSS
-

86. Interview Preparation Roadmap

Month 1

- React basics
- Components
- Hooks
- Routing

Month 2

- Redux
- APIs
- Forms
- Authentication

Month 3

- Optimization
 - Machine coding
 - Testing
 - System design basics
-

87. Final Notes

To crack React interviews for 0–3 years experience:

- Strong JavaScript fundamentals are mandatory
 - Understand hooks deeply
 - Build real-world projects
 - Learn optimization techniques
 - Practice machine coding rounds
 - Know architecture decisions
 - Learn React ecosystem tools
-

88. Quick 50 Interview Questions

1. What is React?
2. What is JSX?
3. What is Virtual DOM?
4. Difference between state and props?
5. What are hooks?
6. Explain useEffect.
7. Explain useMemo.
8. Explain useCallback.
9. What is Context API?
10. What is Redux?
11. Difference between controlled and uncontrolled components?
12. What are keys in React?
13. What is reconciliation?
14. What is React Fiber?
15. What is lazy loading?
16. What is Suspense?
17. What is prop drilling?
18. What is lifting state up?
19. What is memoization?
20. Difference between SSR and CSR?
21. What is hydration?
22. What are portals?

23. What are HOCs?
 24. What is render props?
 25. What is custom hook?
 26. What is React.memo?
 27. Difference between useRef and state?
 28. What are synthetic events?
 29. What is batching?
 30. Explain React lifecycle.
 31. Difference between class and functional components?
 32. Explain dependency array.
 33. What is stale closure?
 34. What are pure components?
 35. What is code splitting?
 36. What is debouncing?
 37. What is throttling?
 38. Explain authentication flow.
 39. What is JWT?
 40. What are protected routes?
 41. Explain Redux flow.
 42. What is middleware?
 43. What is optimistic update?
 44. What is React Query?
 45. Explain reconciliation algorithm.
 46. What causes re-render?
 47. Explain event delegation.
 48. What is hydration mismatch?
 49. How to improve performance?
 50. Explain your project architecture.
-

End of React Interview Guide